프론트엔드 기능 정리

이 문서는 Vibe Coding 화면에서 사용자가 할 수 있는 일을 쉽게 정리한 문서입니다. 내부 파일명이나 구현 세부사항보다 실제 화면 흐름과 기능 중심으로 설명합니다.

한눈에 보기

Vibe Coding의 화면은 크게 다섯 부분으로 나뉩니다.

영역	할 수 있는 일
로그인 화면	로그인, 회원가입
왼쪽 사이드바	채팅 목록 보기, 검색, 새 채팅 만들기, 노트 열기, 설정 열기
채팅 화면	질문 입력, AI 응답 확인, 메시지 복사/수정/삭제
입력창	메시지 작성, 파일 첨부, 실행 방식 선택, 응답 중단
미리보기 패널	생성된 코드를 확인하고 브라우저에서 바로 실행

사용한 기술

프론트엔드는 별도 빌드 도구나 프레임워크 없이 브라우저 기본 기능과 몇 가지 CDN 라이브러리로 구현되어 있습니다.

용도	사용 기술	구현 방식
화면 구성	HTML, CSS, Vanilla JavaScript	정적 HTML 화면에 JavaScript로 상태와 이벤트를 연결합니다.
서버 통신	Fetch API	로그인, 채팅 저장, AI 응답 요청을 HTTP API로 보냅니다.
스트리밍 응답	ReadableStream, TextDecoder	서버에서 오는 응답 조각을 읽어 실시간으로 채팅 말풍선에 붙입니다.
요청 중단	AbortController	답변 생성 중 사용자가 중단하면 진행 중인 fetch 요청을 취소합니다.
상태 저장	localStorage, 서버 저장 API	브라우저에 먼저 저장하고, 로그인 상태에서는 서버에도 동기화합니다.
파일 첨부	FileReader	사용자가 고른 파일을 브라우저에서 읽어 전송 가능한 데이터로 만듭니다.
코드 하이라이트	highlight.js, 내부 폴백 하이라이터	라이브러리가 있으면 사용하고, 없으면 간단한 자체 표시를 사용합니다.
수식 표시	KaTeX	응답 안의 수식을 브라우저에서 렌더링합니다.
다이어그램 표시	Mermaid	Mermaid 코드 블록을 SVG 다이어그램으로 변환합니다.

용도	사용 기술	구현 방식
코드 미리보기	iframe, srcdoc	생성된 코드를 HTML 문자열로 조립해 격리된 실행 영역에 넣습니다.
React 미리보기	Babel Standalone, React, ReactDOM	JSX/TSX를 브라우저에서 JavaScript로 변환해 실행합니다.
반응형 UI	CSS media query	모바일 화면에서는 사이드바와 미리보기 패널 배치를 바꿉니다.

기본 사용 흐름

1. 사용자가 로그인합니다.
2. 이전 채팅과 설정이 자동으로 복원됩니다.
3. 새 채팅을 만들거나 기존 채팅을 선택합니다.
4. 원하는 앱이나 코드 작업을 자연어로 입력합니다.
5. 필요하면 이미지, PDF, 텍스트 파일 등을 첨부합니다.
6. AI 응답이 실시간으로 표시됩니다.
7. 응답 안에 코드가 있으면 복사하거나 미리 실행해 볼 수 있습니다.
8. 채팅 내용은 자동으로 저장됩니다.

채팅 관리

- 새 채팅을 만들 수 있습니다.
- 채팅 제목은 첫 메시지를 기준으로 자동 생성됩니다.
- 채팅 목록에서 제목과 내용으로 검색할 수 있습니다.
- 중요한 채팅은 고정할 수 있습니다.
- 채팅 이름을 바꿀 수 있습니다.
- 채팅을 마크다운 문서로 내보낼 수 있습니다.
- 필요 없는 채팅은 삭제할 수 있습니다.

구현 방식:

- 채팅 목록, 현재 선택된 채팅, 설정값을 하나의 브라우저 상태 객체로 관리합니다.
- 사용자가 채팅을 만들거나 수정하면 먼저 브라우저 저장소에 저장합니다.
- 일정 시간 뒤 서버 저장 API를 호출해 같은 내용을 서버 파일에도 저장합니다.
- 검색은 서버에 다시 요청하지 않고, 이미 불러온 채팅 제목과 메시지 내용을 브라우저에서 필터링합니다.

메시지 기능

사용자 메시지에서 할 수 있는 일:

- 복사
- 수정 후 다시 생성
- 삭제

AI 메시지에서 할 수 있는 일:

- 복사

- 다시 생성
- 삭제

AI가 답변 중일 때는 전송 버튼이 중단 버튼처럼 동작합니다. 긴 답변을 받다가 멈추고 싶을 때 사용할 수 있습니다.

구현 방식:

- 메시지는 사용자 메시지와 AI 메시지를 구분해 화면에 그립니다.
- AI 응답은 한 번에 다 받은 뒤 표시하는 방식이 아니라, 서버에서 도착하는 조각을 순서대로 붙여 표시합니다.
- 생성이 끝나면 실시간 표시용 말풍선을 제거하고 최종 메시지 하나로 저장합니다.
- 복사 기능은 브라우저 클립보드 API를 사용합니다.

실행 방식

입력창 옆에서 실행 방식을 고를 수 있습니다.

방식	설명
확인 후 진행	먼저 요청을 분석한 뒤 코드 생성을 이어서 진행합니다. 현재 화면에서는 명세 확인 카드를 따로 띄우지 않고 자동으로 다음 단계로 넘어갑니다.
바로 진행	요청을 보내면 분석부터 코드 생성까지 한 번에 진행합니다.

두 방식 모두 최종 답변은 채팅 메시지로 저장됩니다.

구현 방식:

- `확인 후 진행`은 먼저 명세 생성 API를 호출한 뒤, 이어서 코드 생성 스트림 API를 호출하는 2단계 구조입니다.
- `바로 진행`은 하나의 통합 스트림 API를 호출하는 1단계 구조입니다.
- 서버가 중간 상태 표시용으로 보내는 문자열은 본문과 분리해 처리합니다.

파일 첨부

입력창에서 파일을 첨부할 수 있습니다.

지원하는 첨부:

- 이미지
- PDF
- 텍스트 파일
- 마크다운 문서

이미지는 작은 미리보기로 표시됩니다. 문서 파일은 파일명으로 표시되며, 전송 전에 개별 삭제할 수 있습니다.

현재 모델 설정에 따라 이미지 내용을 실제로 이해하지 못할 수 있습니다. 이 부분은 코드 리뷰 문서의 개선 항목에도 정리되어 있습니다.

구현 방식:

- 파일 선택 창에서 여러 파일을 받을 수 있습니다.
- 브라우저가 파일을 읽어 데이터 URL 형태로 보관합니다.

- 이미지는 작은 썸네일로 보여주고, 문서 파일은 파일명으로 보여줍니다.
- 전송 시 메시지 내용과 첨부 데이터가 함께 서버로 전달됩니다.

답변 표시

AI 답변은 읽기 좋게 자동 정리됩니다.

- 제목, 목록, 굵은 글씨 같은 기본 마크다운을 표시합니다.
- 코드 블록은 보기 좋게 색을 입혀 표시합니다.
- 수식이 있으면 수식 형태로 보여줍니다.
- 다이어그램 문법이 있으면 다이어그램으로 보여줍니다.
- 링크는 새 창에서 열리도록 처리합니다.

구현 방식:

- AI 응답 문자열을 먼저 마크다운 형태로 분석합니다.
- 코드 블록은 일반 본문과 분리해 별도 코드 박스로 만듭니다.
- 코드 언어를 확인해 하이라이트와 미리보기 가능 여부를 결정합니다.
- 수식과 다이어그램은 응답이 화면에 들어온 뒤 후처리로 렌더링합니다.

코드 미리보기

AI가 만든 코드에는 미리보기 기능이 붙습니다.

할 수 있는 일:

- 코드 복사
- 단일 코드 실행
- 여러 파일로 구성된 코드 전체 실행
- 미리보기 패널에서 코드 수정
- 수정한 코드를 자동으로 다시 실행

HTML, CSS, JavaScript, React 계열 코드처럼 브라우저에서 실행 가능한 코드는 바로 확인할 수 있습니다.

서버 코드나 모바일 앱 코드처럼 브라우저에서 직접 실행하기 어려운 코드는 안내 화면을 보여줍니다.

미리보기 패널

미리보기 패널은 코드와 실행 결과를 함께 보여줍니다.

- 왼쪽에는 코드가 표시됩니다.
- 오른쪽에는 실행 결과가 표시됩니다.
- 여러 파일이 있으면 파일 탭을 바꿔가며 볼 수 있습니다.
- 코드를 수정하면 잠시 뒤 자동으로 다시 실행됩니다.
- 패널 크기와 코드/실행 영역의 비율을 조절할 수 있습니다.

생성된 코드는 분리된 실행 영역에서 동작합니다. 다만 AI가 만든 코드는 항상 신뢰할 수 있는 것은 아니므로, 중요한 환경에서는 그대로 실행하지 않는 것이 안전합니다.

미리보기 구현 방식

미리보기는 AI 답변 안의 코드 블록을 읽어서 브라우저가 실행할 수 있는 임시 HTML 문서를 만드는 방식으로 동작합니다. 실제 파일을 디스크에 저장하거나 개발 서버를 띄우지 않습니다.

전체 흐름은 다음과 같습니다.

1. AI 답변에서 코드 블록을 찾습니다.
2. 코드 블록의 언어와 파일 이름 정보를 읽습니다.
3. 실행 가능한 코드인지 판단합니다.
4. 코드가 하나뿐이면 단일 문서로 만듭니다.
5. 여러 파일이 있으면 하나의 작은 프로젝트처럼 묶습니다.
6. 최종 HTML 문자열을 만듭니다.
7. 이 문자열을 iframe의 `srcdoc`에 넣어 실행합니다.
8. 사용자가 코드를 수정하면 잠시 뒤 다시 HTML 문자열을 만들어 실행합니다.

HTML, CSS, JavaScript로 구성된 코드는 다음처럼 처리합니다.

- HTML 파일을 기준으로 실행 문서를 만듭니다.
- HTML 안에서 연결한 CSS 파일을 찾아 스타일 태그로 합칩니다.
- HTML 안에서 연결한 JavaScript 파일을 찾아 스크립트 태그로 합칩니다.
- 이미지나 외부 자산처럼 찾을 수 없는 파일은 깨지지 않도록 가능한 범위에서 정리합니다.
- 이렇게 합친 HTML 문자열을 iframe에 넣어 실행합니다.

React 계열 코드는 조금 다르게 처리합니다.

- 여러 파일을 브라우저 안의 가상 파일 묶음으로 관리합니다.
- import 문으로 연결된 파일을 찾아 실행 순서를 맞춥니다.
- JSX나 TSX 문법은 Babel Standalone으로 브라우저에서 실행 가능한 JavaScript로 변환합니다.
- 필요한 경우 React와 ReactDOM을 브라우저에서 불러와 실행합니다.
- Tailwind 사용이 감지되면 Tailwind CDN을 함께 넣습니다.

미리보기 실행 영역은 일반 페이지와 분리되어 있습니다.

- 코드가 현재 앱 화면을 직접 바꾸지 못하도록 sandbox가 적용된 iframe을 사용합니다.
- 저장소 기능은 미리보기 안에서만 동작하는 임시 저장소로 대체됩니다.
- 링크 클릭이나 폼 제출로 앱 화면을 벗어나지 않도록 막습니다.
- 새 창 열기나 강제 이동 같은 동작도 제한합니다.

코드 편집 후 다시 실행되는 방식은 다음과 같습니다.

- 미리보기 패널의 코드 영역은 브라우저 안의 간단한 텍스트 편집기입니다.
- 사용자가 코드를 수정하면 바로 실행하지 않고 아주 짧게 기다립니다.
- 입력이 잠시 멈추면 현재 코드 묶음으로 HTML 문서를 다시 만듭니다.
- 기존 iframe을 새 iframe으로 교체해 이전 실행 상태를 정리합니다.

설정

설정 화면에서 다음을 관리할 수 있습니다.

- 화면 테마: 다크, 라이트, 시스템 설정
- 전체 채팅 내보내기
- 사용자 기억 관리

사용자 기억은 AI가 이후 요청에서 참고할 수 있는 짧은 메모입니다. 예를 들어 선호하는 기술 스택이나 작업 방식 같은 내용을 저장할 수 있습니다.

개인 노트

노트 화면은 사용자가 자유롭게 메모를 적는 공간입니다. 채팅과 별도로 보관되는 개인 메모로 이해하면 됩니다.

모바일 화면

작은 화면에서는 사이드바가 모바일 메뉴처럼 열리고 닫힙니다.

입력창은 모바일 브라우저에서 과도하게 확대되지 않도록 조정되어 있습니다.

자동 저장

채팅, 설정, 노트 일부 상태는 브라우저에 먼저 저장됩니다. 로그인 상태에서는 서버에도 함께 저장되어 다음 접속 때 이어서 사용할 수 있습니다.

브라우저 저장 공간이 부족하면 현재 채팅과 설정처럼 꼭 필요한 정보만 우선 저장하려고 시도합니다.